

Vysoká škola ekonomická v Praze

Fakulta informatiky a statistiky



· B·E·H·A·V·I·O

ZÁVĚREČNÁ ZPRÁVA DATOVÉHO PROJEKTU

Tým 6_Null Hypothesis – P7_Behavio DWH

Studijní program: Aplikovaná datová analytika a umělá inteligence

Autoři: Martin Kadlec, Marek Bezvoda, Michael Nguyen, Jakub Hájek, Erika Žáčiková

Mentor Projektu: Miroslav Umlauf

Praha, červen 2025

1 Úvod

Datový projekt je realizován ve spolupráci se společností Behavio, též známou pod doménou Behaviolabs. Jedná se o firmu specializující se na marketingový výzkum se zaměřením na behaviorální analýzu.

Firma v horizontu posledních dvou let přešla do stádia rapidního růstu, během kterých se z dřívější defacto startupové společnosti stává seriózním hráčem na poli globálního marketingového trhu, a to díky spuštění SaaS platformy, která umožňuje rychlou a poměrně jednoduchou integraci do prostředí (nejen) e-commerce poskytovatelů za účelem sběru, analýzy a vyhodnocení relevantních marketingových ukazatelů.

Tento rozvoj s sebou jednak přináší nové výzvy, co se týče operativy spojené s řízením klientů a jejich jednotlivých projektů, ale také je spojen s příchodem externího kapitálu do firmy, kde se z investorů stává regulérní stakeholder chodu a stavu firmy, což znamená rozšíření nároků na informační a rozhodovací workflows ve firmě.

Z těchto důvodů vznikla poptávka po modernizaci interního reportingového systému. Společnost Behavio začala ve spolupráci s externími konzultanty realizovat prototyp interního datového skladu, kde začíná postupně konsolidovat výstupy z primárních transakčních systémů. Technologie tohoto DWH je BiqQuery v rámci Google Cloud Platform (GCP). Záměrem datového projektu je navržení konceptu, který by rozšířil tento GCP DWH o komponenty zajišťující automatizaci dohledu nad klíčovými metrikami, umožnily rychlejší extrakci insights z dat v DWH a poskytly substrát pro další rozvoj DWH.

Na základě vzájemné diskuze se zástupcem společnosti Behavio – CFO Lukášem Tóthem byly identifikovány tři klíčové domény/cíle výsledného řešení:

- Realizace automatického (ve smyslu periodické/trigger based kontroly) alertingového mechanismu, který by eliminoval rizika vzniku kontrafaktuálních stavů a rozhodnutí vni firmy
- Navržení konceptu pro Natural Language Querying (v textu nadále jako NLQ) stavu DWH, ve smyslu realizace chatbota se schopností interagovat s DWH a podávat business uživatelům rychlé a přesné informace bez nutnosti sestavování vlastní analýzy, a to především se záměrem zrychlit reakce na podněty investorů
- Realizace libovolných podpůrných kroků nutných k realizaci předchozích dvou bodů (tj. úprava či rozšíření DWH struktury, integrace rozšiřujících komponent a potřebných konfigurací na straně GCP, patřičné zásahy do analytické/prezentační vrstvy)

Výsledné řešení musí v obecné rovině minimalizovat náklady a maximalizovat udržitelnost, a to především tak, aby řešení nevyžadovalo zvýšený overhead na personální obsluhu na straně Behavio.

3 Architektura řešení

Pro potřeby závěrečné zprávy jsou rozlišeny tři architektonické pohledy: souhrnný pohled, architektura alertingového řešení a architektura LLM komponenty.

3.1 High-level perspektiva

Jak již bylo nastíněno v podkapitole Definice stavu „as is“, datový projekt nezačíná na zcela zelené louce a opírá se o dřívější snahy ze strany Behaviolabs budovat prostor pro konsolidaci svých dat. Stav tohoto řešení předepisuje iniciační technologický rámec a zároveň určuje nutný výchozí integrační bod. Řešení požadovaných UC je závislé na vhodné integraci s BigQuery. Rešeršní a eliminační postup je k dispozici v podkapitole Proces volby vybraných technologií.

Výsledné řešení se opírá o možnosti komponenty Cloud Run, která umožňuje v prostředí GCP využívat vlastní a modifikovatelné compute resources. Pro potřeby datového projektu je použito programové vybavení Pythonu a odpovídající Google client APIs.

Tyto komponenty se mohou odkazovat – skrze delegované servisní účty – na obsah BigQuery. Za účelem optimalizace a dodržování best practice přístupů je vhodné za tímto účelem konfigurovat konkrétně odvozené views⁹.

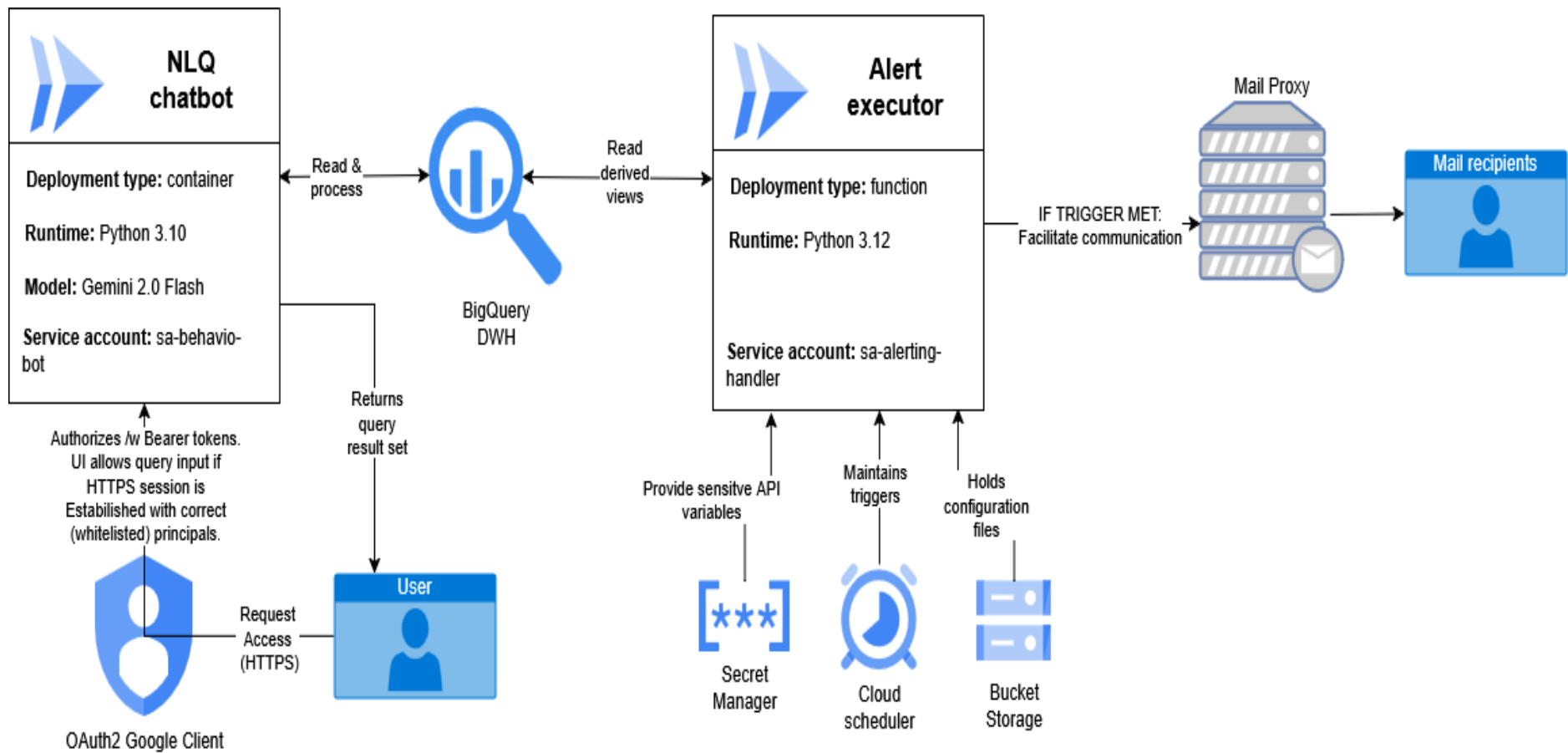
Přístup k NLQ komponentě je dostupný na standardním HTTPS portu 443 díky možnostem cloud run platformy. V době odevzdávání práce je pro předávku alertů používán mailový kanál, mail zprostředkovatel je Mailgun¹⁰.

K NLQ chatbotu se lze dostat pouze na základě úspěšné autentizace vůči Google OAuth klientu (chatbot má ve své aplikační vrstvě integrovaný whitelist). Mailing list alertů je součástí konfigurace obsluhující Cloud Run funkce, která je opět vyskláděna vni GCP. Platforma tedy neposkytuje citlivé informace ve vnějších rozhraních a systém je zabezpečen vůči neoprávněnému přístupu.

Model architektury je zachycen na samostatné stránce:

⁹ Toto je důležité především pro limitování result setů pro alerting. U NLQ je diskutabilně ten správný přístup čist „celý“ DWH. Z důvodů blíže popsanych v konkrétní implementační kapitole je však vhodné limitovat přístup pouze na určité části datového skladu tak, aby model správně pracoval s kontextem a aktivními záznamy v řešení.

¹⁰ A to díky příznivému free-tier plánu, komprehensivní diskuze je v implementačních kapitolách. Některým čtenářům může připadat volba mailu jako primární platformy kuriozní – oproti např. webhooks do Slacku či jiné chat platformy. Požadavek byl vydefinován v rámci řešení implementačních možností – do kolaborační platformy Behavio nemají někteří externí stakeholderi alertingu přístup a ani to není žádoucí.



3.2 Alerting

Alertingová komponenta je řešena jako kombinace tří GCP komponent – Cloud Run Functions, Cloud Scheduler a BigQuery¹¹. Tato volba umožňuje kompletní programovou kontrolu nad chováním alertu a jeho distribučním mechanismem. GCP kvůli striktním bezpečnostním nárokům vyžaduje využití mail proxy mimo compute prostředí GCP stacku. Pro potřeby MVP je tedy tato funkce řešena přes Mailgun API¹².

Na úrovni BigQuery došlo k předzpracování klíčových indikátorů do podoby SQL views, architektonicky chápeme jednotlivé „alert cases“ jako jeden ucelený insight zachycený a vyjádřitelný jako SQL query v kardinalitě 1:1¹³.

Nad BigQuery jsou pak v Google Cloud Scheduleru vydefinovány potřebné periody kontrol (dle potřeby business casu), které aktivují Google Cloud Run funkci, která díky modulárnímu konfiguračnímu souboru provede kontrolu záznamů propsaných do připravených views, a v případě naplnění podmínek aktivační funkce dojde k odeslání mailu nakonfigurovaným příjemcům. V současné implementaci pracují v tandemu dvě periody. Jedna slouží pro pravidelný souhrnný reporting, zatímco druhá je trigger-based a spouští se častěji na základě konkrétních událostí nebo změn ve zdrojových datech.

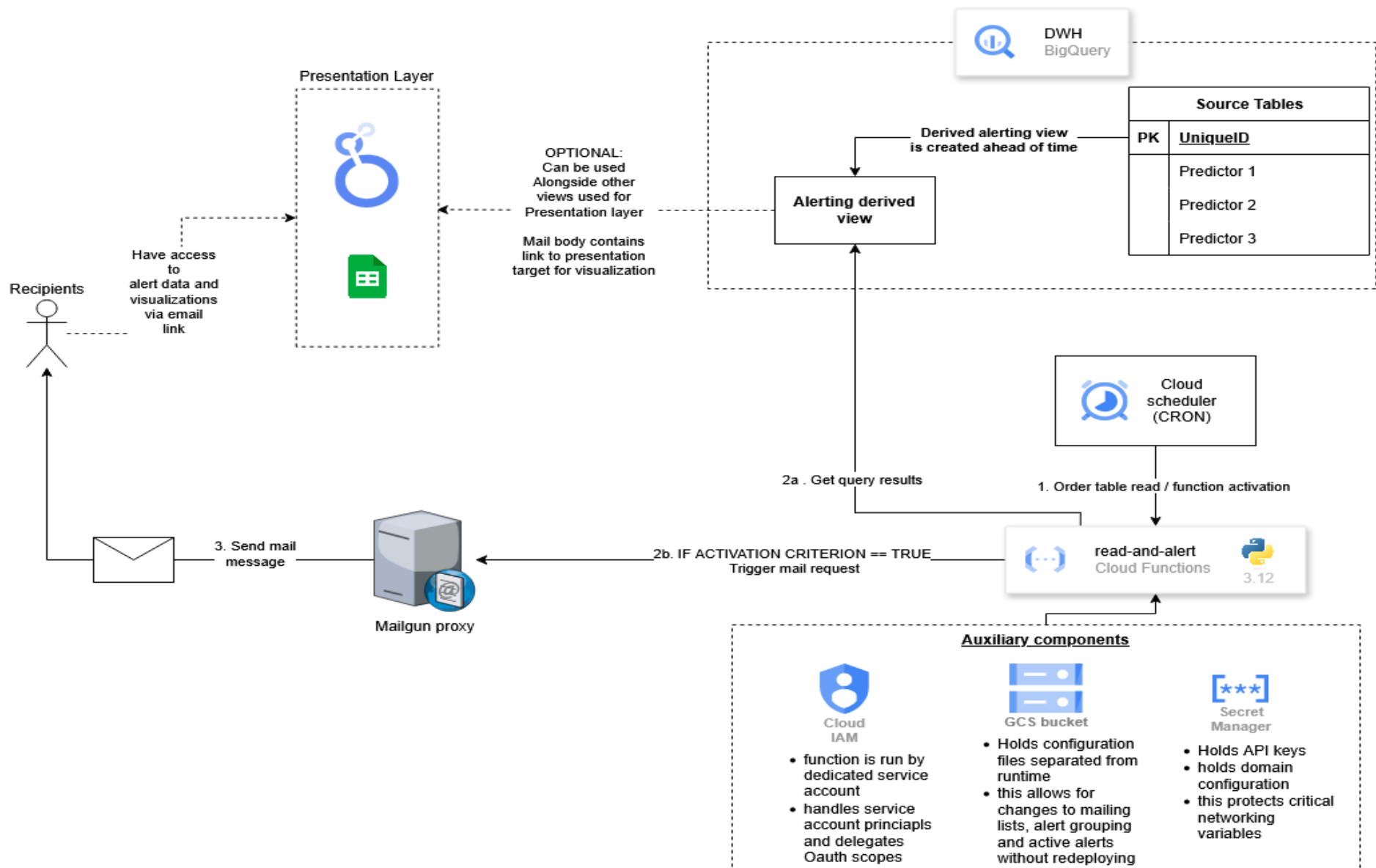
Je vhodné uvést, že vzhledem k neexistující automatizaci ingestu z primárních systémů bude vhodné pro potřeby produkčního systému používat nižší periodu pro event based kontrolu v zájmu optimalizace nákladové funkce stacku. Cloud Scheduler používá CRON mechanismus.

Model je zachycen na následující straně:

¹¹ Použití BigQuery je pochopitelně nutná prerekvizita vzhledem k business kontextu projektu.

¹² Řešení může na první pohled působit těžkopádně, kompletní odůvodnění voleb pro MVP je v subkapitole Proces volby vybraných technologií. Pro potřeby architektonického pohledu je vhodné uvést již teď, že volba mailing klienta je netriviální záležitost, s odkazem na oficiální dokumentaci (<https://cloud.google.com/compute/docs/tutorials/sending-mail>), GCP velmi striktně limituje standardní SMTP komunikaci a oficiálně doporučuje použití mailing agenta třetí strany. Tato volba může působit kuriózně ve světle toho, že např. AWS podporu pro mail klienta má nativně. GCP ji podporuje pouze pro Google Workspace uživatele (tj. placené verze Google účtů určené pro firmy) a to skrze konfiguraci Gmail API, tato konfigurace je pak ale ve výsledku podobná nastavení Mailgun API – je potřeba explicitně definovat externí SMTP server nebo doplnit potřebné DNS záznamy.

¹³ Není to nutností, je to praktické z konfiguračních a pragmatických důvodů – tento přístup mj. umožňuje modulární kontrolu přístupu k jednotlivým pohledům a jejich rychlou rekombinaci při změně recipient listu, views lze také v tomto přístupu rychle exportovat do prezentační vrstvy (Looker Studio, Google Sheets).



3.3 LLM chatbot / NLQ komponenta

Chatbot je implantován pomocí programového vybavení Python, konkrétně jsou využity Google klientské knihovny pro BigQuery, VertexAI a OAuth.

Jako vybraný model se používá architektura Gemini Flash. Aplikační logika je zabalena do knihovny Streamlit – která zajišťuje frontendové funkcionality – a jako jeden modul je zapouzdřena do Docker kontejneru, který lze nasadit do prostředí Google Cloud Run.

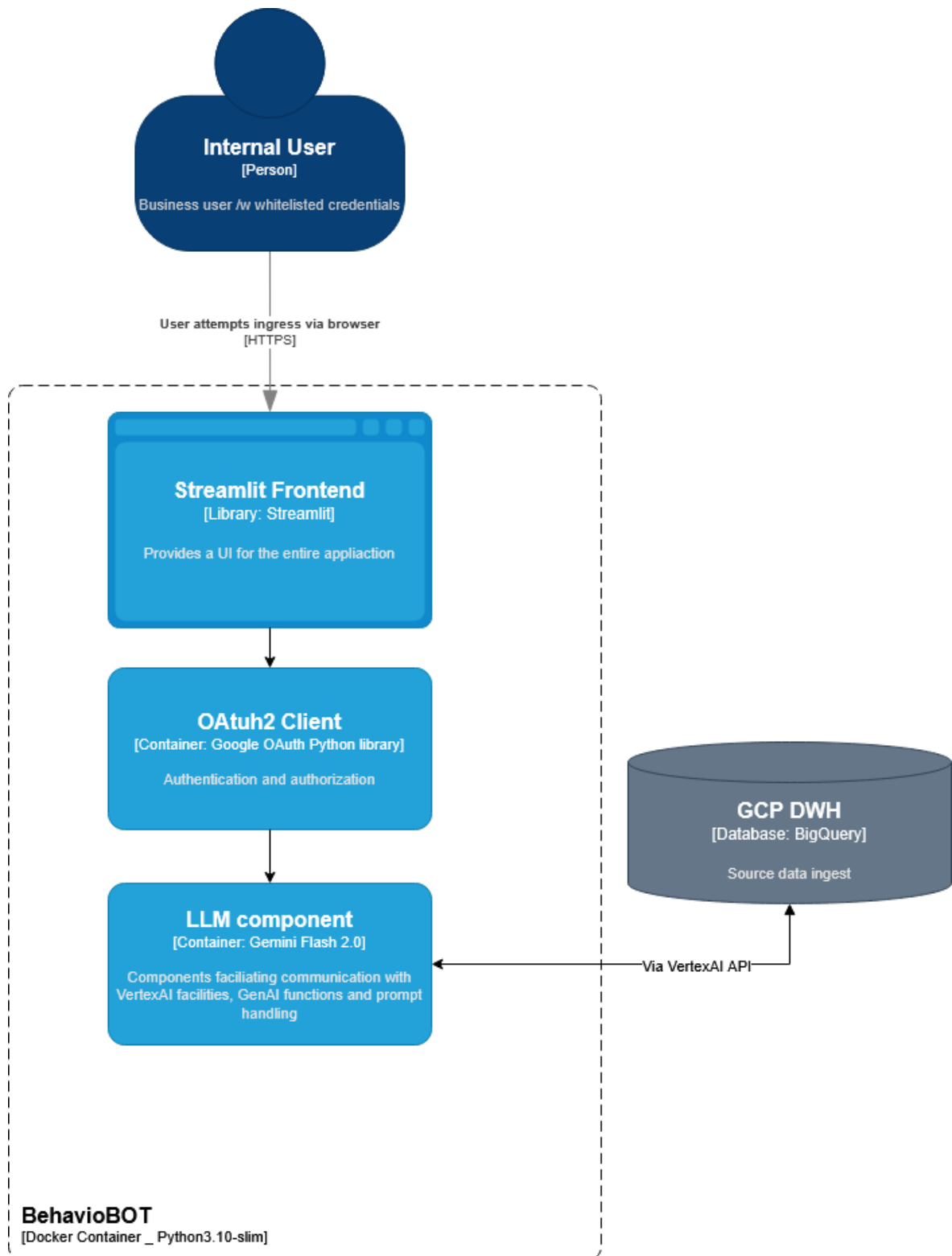
Aplikace je zpřístupněna skrze standardní HTTPS ingress jako webová služba. K zajištění toho, aby data Behaviolabs nebyla přístupná veřejně, je celá aplikace schována za Google OAuth klienta, který používá whitelist metodu vůči výčtu konkrétních uživatelských mailů.

Architektonická orchestrace je přenechána v GCP autoscaling režii. Celá aplikace je tedy uspávána v situacích, kdy vůči ní nejsou vedeny aktivní requesty. Veškerá webová komunikace je logována, včetně identifikace uživatelů aplikace.

Orchestrace LLM spoléhá na nativní funkcionality Gemini Flash modelu a korekce funkcionality je řízena přes Prompt Engineering. Jako kontextové okno se používá kombinace:

- Initial system promptu s instrukcemi pro chatbota
- Kontext stavu DWH (metadata, popis tabulek a sloupců)
- Historie konverzace
- Uživatelský dotaz

Model je zodpovědný za tvorbu a validaci SQL dotazu a zpracování result setu, který je získán z BQ DWH. V případě úspěšného zpracování dotazu se výstupy převádějí na Pandas DataFrame, který je pak serializován a spolu s kontextem předán zpět Gemini modelu ke generaci shrnutí/odpovědi na původní dotaz.



4 Dokumentace řešení

Veškerý programový kód, včetně pomocných artefaktů je dostupný na:

https://gitlab.com/R4tmax/dp_automateddwh

Součástí repozitáře je i přehledová dokumentace vyskladněná jako wiki:

https://gitlab.com/R4tmax/dp_automateddwh/-/wikis/home

4.1 Proces volby vybraných technologií

Diskutabilně nejdůležitější kapitolou této zprávy je rigorózní diskuze použitých technologií a jejich selekční kritéria.

Jak již bylo nastíněno v předchozích kapitolách, celý projekt je ukotven v potřebě pracovat v GCP a kolem možností BigQuery.

Kontext celého řešení je uchopen tak, že výsledné řešení by v první řadě mělo být jednoduché na údržbu – z tohoto důvodu na začátku projektu existovala jistá preference pro OOTB řešení vni GCP. Google v rámci GCP propaguje své „Jump start“ solutions. Jedná se prepackaged komponenty kolem nějakého konkrétního use case a jedná se patrně o nejrychlejší deployment přístup v rámci celé platformy. V době zahájení projektu (tj. v druhé půli března) Google takto propagoval především různé RAG či Datalake solutions, tj. v té době řešení pro vymezený case neexistovalo.

Výchozí rešerše vedla k závěru, že nejvíc cost-effective přístup je používat možnosti Cloud Run¹⁴ prostředí pro navrhnutí vlastních systémů. Různé komponenty mají z hlediska implementace různá omezení. Naší snahou byla maximalizace vyhovění požadavkům pro dané use cases tak, jak byly diskutovány v průběhu projektu s Lukášem Tóthem z Behaviolabs. Hlavní kritéria, na které tým cílil v návrhu, je cena, náročnost maintenance a míra vyhovění požadavkům od Behaviolabs¹⁵.

4.1.1 Alertingová komponenta

Jako preferovaný distribuční kanál byl vybrán email. V rámci raných diskuzí bylo diskutováno nad možnostmi jiných prostředků (např. Slack), ale od tohoto bylo upuštěno ze dvou důvodů:

¹⁴ Cloud Run umožňuje v prostředí GCP provádět exekuci vlastního kódu (pro potřeby datového projektu Python) v automaticky škálovatelných a hostovaných kontejnerech.

¹⁵ Naším záměrem bylo, aby řešení bylo co nejvíce „fire and forget“ za nejnižší dostupnou provozní cenu a bez zbytečných kompromisů/simplifikací od byznys očekávání.

- Firemní kultura preferuje držet perzistentní/klíčové informace v mailu (kvůli dohledatelnosti)
- U alertingu existuje předpoklad pro distribuci alertů mimo prostředí firmy (typicky investorům), a tedy je vhodné používat distribuci v systému, u které není nutné složitě řídit přístup pro externí uživatele

Výchozí očekávání týmu bylo použití nativních Google možností, ale poměrně záhy se ukázalo že GCP nepodporuje „přímé“ odesílání mailů ze své platformy podobným způsobem, jako například Amazon v rámci AWS. Oficiálně citovaným důvodem jsou bezpečnostní nároky na zajištění SMTP komunikace. Veškerá „mail based“ komunikace z prostředí GCP tedy musí chodit formou SSL zabezpečených kanálů, a oficiální doporučení je používat zprostředkovatele API třetích stran¹⁶.

Přirozenou reakcí na tuto informaci je konstatování faktu, že Google provozuje vlastní mailing službu Gmail. Vskutku, GmailAPI je jedna z možností řešení mailové komunikace, ale její použití v prostředí GCP je problematické – systém nedovoluje nastavit stabilní přístupový token pro soukromé uživatele, tato možnost je přístupná pouze Google Workspace uživatelům a konfigurace vyžaduje spolupráci s adminem daného workspace. Z tohoto důvodu řešitelský tým upustil od použití GmailAPI pro potřeby datového projektu, ale doporučuje tento přístup v rámci Doporučení pro další rozvoj a pro použití v produkci.

Alternativních řešení je několik.

První z nich je použití OOTB funkcionalit v prostředí Looker Studio. Konkrétně je alerting poskytnut jako funkcionalita v placeném Pro plánu. V systému alerting funguje tak, že na úrovni jednoho dashboardu se vydefinovala agregační aktivační podmínka, která při naplnění automaticky předá odkaz na dashboard prostřednictvím Gmail nebo Google Chat. Looker Pro plány stojí 9 USD / month per user a umožňují alertovat pouze v prostředí Google Workspace¹⁷. Toto porušuje dříve vydefinované očekávání a jednoduchost řešení je vykoupena poměrně příkrou cenou¹⁸.

Google dále umožňuje používat low-code platformu Google Apps Script, velmi jednoduše řečeno se jedná o odpověď na Microsoft Power Apps. Implementační jazyk je JavaScript a umožňuje konfiguraci např. na úrovni Looker Studio. Aplikačně se jedná stejný princip, který se používá v rámci současně dodaných Python funkcí, ale Apps Scripty nelze tak jednoduše konsolidovat v GCP, z tohoto důvodu tým od tohoto přístupu ustoupil.

Z výše popsaných důvodů tým v řešení pokračoval doporučenou cestou z dokumentace. Finální vybranou službou je Mailgun, který poskytuje stabilní free tier na sto emailů denně. Tato služba by realisticky dokázala pokrýt i očekávané produkční zátěže.

¹⁶ Google propaguje řešení od společnosti SendGrid.

¹⁷ Google Workspace účet je nutnou prerekvizitou aktivace Pro plánu. Jako usera chápeme uživatele s vyšší licenci, který má právo zakládat aletry. Příjemce nemusí být držitel pro plánu, ale musí být ve stejném Google Workspace.

¹⁸ V porovnání s running costs dodaného řešení Datového Projektu.

4.1.2 NLQ komponenta

V rámci rešerše a výběru NLQ komponenty prvotní kroky směřovaly k použití VertexAI modulů vni samotného GCP. Z konfiguračních důvodů však nelze komponenty vhodně nastavit – protože funkcionality nejsou v plném rozsahu poskytovány na evropských serverech.

Jelikož klíčovou prerekvizitou je možnost přístupu na BigQuery takovým způsobem, který vyhovuje bezpečnostním požadavkům pro produkční systémy, z tohoto důvodu je vhodné držet se Google ekosystému.

V době řešení projektu v GCP existuje například komponenta Dataplex, která byla vyvinuta jako nástroj pro data governance. Součástí této komponenty je i tzv. Data Canvas, který se limitně hodně blíží očekávaný nárokům na NLQ řešení. Po rešerši dokumentace a konzultaci s mentory bylo však od tohoto řešení upuštěno, protože není navrženo pro použití business uživateli – stále je potřeba provozovat jistou míru vlastního zpracování dat a řešení tedy není vhodné pro „nekvalifikované“ SSBI¹⁹. Použití Dataplexu by zároveň zvýšilo running costs²⁰.

Řešení je opět použití vlastního runtime pro obsluhu aplikační implementace.

K VertexAI modelům lze přistupovat pomocí specializovaného API, a skrze oficiální Google Python knihovny lze deklarovat potřebné objekty přímo ve vlastním kódu. S tímto přístupem tedy lze kombinovat možnosti Vertex modelů (Gemini, Palm, Gemma) s vlastním programovým kódem.

Toto je velmi výhodný přístup, protože umožňuje model kombinovat s vlastním LLM orchestračním řešením – především manipulaci s kontextem a dodáním potřebných toolů.

Kritickou otázkou pak je, který model pro řešení použít.

Jako odpověď lze použít současné benchmark výstupy dostupných LLM. Aktuální stav benchmarků byl prezentován v keynote Google Cloud Summit 25 ([záznam](#)) a letošní NEXT konferenci. Dle oficiálního Google narativu je Gemini Flash nejvíce cost-effective model momentálně dostupný na trhu. V dnešním ekosystému existuje velká míra skepse ohledně kvality a validity prováděných benchmarků, ale na základě rešerše a testování řešitelského týmu se tvrzení jeví poměrně validní. Klíčovou výhodou Flash modelu je kromě jeho dostupnosti ve VertexAPI jeho multimodalita, která napomáhá správným NL-SQL konverzním potřebným pro realizaci datového projektu. Další velkou výhodou je konformita k LangChain standardu, který zjednodušuje LLM orchestraci.

¹⁹ Myšleno např. pro použití sales personálem.

²⁰ Je vhodné uvést, že Dataplex byl krátce diskutován i jako vhodné řešení pro alerting – v rámci komponenty existuje monitoring, který umožňuje dále skrze GCP delegovat alerty. Po iniciálním prozkoumání této možnosti se však systém jeví jako krkolomný a jeho implementace by vyžadovala velké ohnutí funkcionalit od jejich původního záměru.

Poslední otázkou je pak deployment model. Jak již bylo zmíněno v architektonické části, pro dodání frontendu do řešení je použita knihovna Streamlit, která je navržena pro rychlý návrh data science aplikací a jejich distribuci.

Primární distribuční metoda je pak Docker. Google Run kromě poskytování BE pro vlastní runtimes umožňuje i deployment kontejnerů. Ironií osudu toto nebyla původně testovaná varianta v rámci datového projektu – kde na doporučení mentorů se původně zvažoval systém App Engine. Na základě testování se však ukázalo, že pro Google je toto řešení převážně deprecated, a dnešní preferovanou metodou je použití Google Cloud Run modelu.

Klíčovou výhodou je zde autoscaling a automatické řízení webového přístupu. Cloud Run kontejnery jsou by default přístupné jako webové aplikace a by default jsou automaticky orchestrovány a škálovány podle objemu příchozí komunikace. Kontejnery lze v případě nulového traffiku uspat na nulový počet instancí, za které pochopitelně nenabíhá žádný billing, což je perfektní pro zde řešený use case.

Přístup přes veřejný internet pochopitelně představuje bezpečnostní riziko. Toto je řešeno přidáním OAuth klienta do aplikačního kódu. Veškerý http provoz směřující do aplikace tedy musí nejdříve projít přes standardní Google SSO, na úrovni OAuth klienta lze definovat kompletní whitelist, který byl nastaven na Behaviolabs doménu, aplikace je tedy nedostupná pro kohokoliv jiného.

4.1.3 Assessment kompromisů v tech stacku

Jistým pohledem jako hlavní kompromis v předloženém řešení lze vnímat deviaci od používání OOTB postupů. Na začátku projektu toto bylo do jisté míry očekávané řešení problému, ale na základě postupné analýzy a testování požadavků se toto ukázalo jako nevhodné či dokonce místy zcela nevyhovující řešení. Kompromis tedy pramení v záměně pragmatičnosti použití hotových řešení za modularitu řešení nakonec dodaného. Vykupující proměnná je konfigurační komplexita, která je však izolována do iniciálního nastavení komponent a nepředstavuje provozní zátěž.

Ryze technickým pohledem je pak největší kompromis v odchylce od best practices v deployment postupu. V ideálním případě jsou Cloud Run artefakty, tedy dodané výstupy, nasazovány do GCP prostředí pomocí přímé CI/CD integrace mezi poskytovatelem Git²¹ služby a GCP projektem. Tímto přístupem je kromě všech standardních VCS a CI/CD výhod dodržena i elementární config/source code separace, jelikož lze konfiguraci editovat mimo běh funkce a orchestrovat její redeployment. Pohledem na druhou stranu mince lze toto poněkud překvapivě interpretovat i jako výhodu, Behavio dle jejich vlastních tvrzení preferuje „neprogramátorské“ řešení a při představení tohoto stavu během UAT nijak nenamítalo. Naopak Lukáš Tóth poznamenal, že je tato varianta vcelku únosná.

²¹ Existují connectory pro GitHub, GitLab i Bitbucket. GitHub je platforma s největší podporou. Realisticky lze problém dekomponovat do úrovně shell skriptů či YAML workflows a neměl by být větší problém používat i self-hosted repozitáře.

Jako další kompromis lze v řešení chápat i současný stav mailing proxy, která využívá limitovaný free tier. V případě potřeby lze řešení zmigrovat do prostředí Google Workspace za využití GmailAPI jako mailing klienta²². Problematika volby mailing agenta vydá na poměrně komplexní tematiku. Proces konfigurace systému je vyskládněn na [wiki](#) datového projektu. Pragmaticky můžeme chápat doporučení používat GmailAPI oproti současnému Mailgunu jako futureproofing řešení, a to jak infrastrukturní, tak nákladové. Vyvažující mince je větší vendor-dependency a s tím spojená rizika lock-inu.

Jako jistou formu kompromisu lze chápat použití Streamlit knihovny pro NLQ řešení, knihovna se typicky nepovažuje jako „PROD ready“, jelikož má omezené možnosti vlastního přizpůsobení. Pro očekávání daného use case je však plně dostačující a jediná „vada na kráse“ je, že aplikace z principu bude mít vždy proprietární Streamlit komponenty a stylování.

4.2 Implementační postup

Následující kapitoly jsou sestaveny jako průvodce implementačním postupem komponent dodaných v datovém projektu. Jejich původní „minimum data set“ je k dispozici na https://gitlab.com/R4tmax/dp_automateddwh/-/wikis/IMPLEMENTATION.

V relevantních podkapitolách je uveden „breadcrumb“ poukazující na současně řešenou komponentu GCP.

4.2.1 Alerting

Příprava projektu

Komponenty potřebují v instanci mít aktivní APIs:

- Cloud Run API
- BigQuery API
- Cloud Scheduler API
- Secret Manager API
- Cloud Storage API

Dále implementační osoba musí mít v IAM přidělené následující role²³:

²² Konfigurační postup je veskrze podobný, mailing zprostředkovatel musí být nastaven na úrovni DNS záznamu. Poté lze nastavit v rámci Google Workspace „virtuální“ účet používající tzv. *Domain-wide delegation*. Free limity GmailAPI jsou mnohem přívětivější než poskytovatelé třetí strany ale bez správné konfigurace na úrovni Workspace nelze docílit plné funkcionality – velmi ostře jsou např. limitovány nastavení bezpečnostních tokenů, které je potřeba měnit každých 7 dní.

²³ Výčet povolených práv a aktivací API může působit poněkud excesivně, problém je zakotven v tom, že GCP neposkytuje dobré „předpřipravené“ role. Problematika byla aktivně konzultována během běhu projektu a realisticky platí, že pro každý použitý „resource type“ je potřeba aktivovat jeho funkci

- Artifact Registry Administrator
- BigQuery Admin
- Cloud Run Admin
- Cloud Scheduler Admin
- Private Logs Viewer
- Project IAM Admin
- Secret Manager Admin
- Service Account User
- Service Usage Admin
- Storage Admin

Konverze alert cases do SQL

Target breadcrumb: BigQuery

Namísto přímého dotazování na operativní tabulky v datovém skladu byly vytvořeny pohledy (views). Každý alert má vlastní view, které obsahuje přesně definovanou logiku (např. "nezaplacené faktury starší než 30 dní", "překročení rozpočtu o více než 10 %", atd.).

Název ani forma vytvoření tohoto view není důležitá a sleduje klasické SQL konvence. Z pohledu zde odevzdávaného řešení je důležité, že views subsetují poptávané informace tak, aby nebylo nutné skrze pravidelné kontrolní joby načítat celé tabulky. Je vhodné používat jednotnou jmennou konvenci.

Architektonicky chápeme, že jeden alert case odpovídá jednomu SQL view.

Prezentační vrstva alertu

Každý alert má vlastní Google Sheet, který zobrazuje výstup daného view (přes propojení BigQuery → Google Sheets). Tento sheet slouží jako hlavní prostředek pro sdílení výsledků alertů s relevantními uživateli. Opět platí, že jedno view by mělo mít jeden Google Sheet.

Tyto sheets umožňují jednak granulárně řídit přístupy (např. lze takto rozlišit, kdo má právo být upozorněn a kdo má právo být upozorněn a zároveň číst podkladová data). Obsah sheetu načítá obsah result setu přes BQ konektor.

Volba Google Sheetu jako abstrakce pro předávku dat je měnitelná, v rámci dalšího rozvoje je např. možné jako spojující linku používat Looker vizualizace namísto Sheetů, architektonicky je způsob řízení přístupu ekvivalentní.

Při odesílání e-mailového upozornění daného alertu funkce automaticky připojí:

- Textové upozornění (např. „Byly nalezeny 3 nezaplacené faktury po splatnosti“)

vni instance a každé založení nového resource připadá pouze admin roli. „Laciné“ řešení je přidělit implementační osobě roli Editor, ale toto není best practice přístup. Popis očekávaných rolí je vždy v oficiální dokumentaci komponenty a je vhodné se ho držet.

- Odkaz na odpovídající Google Sheet, kde si příjemce může detailně prohlédnout výsledky alertu

Struktura mailového reportu je prezentačně modifikována jako HTML dokument, což umožňuje alertu dodat základní vizuály.

Nastavení Service Agenta pro Cloud Run

Target breadcrumb: IAM & admin / Service accounts

Základem pro správnou funkčnost Cloud Run služby je vytvoření service accountu, kterým Cloud Run autentizuje přístup k dalším GCP službám. Název SA je konfigurační parametr v následujících krocích. SA je pak potřeba v IAM přiřadit následující principals:

Role	Účel
roles/bigquery.dataViewer	Čtení dat z BigQuery
roles/bigquery.jobUser	Spouštění dotazů v BigQuery
roles/run.invoker	Vyvolávání Cloud Run služeb
roles/logging.logWriter	Zápis logů do Cloud Logging
roles/secretmanager.secretAccessor	Přístup k tajným údajům uloženým v Secret Manageru

Separace konfiguračních souborů od zdrojového kódu

Target breadcrumb: Cloud Storage / Buckets

Pro zvýšení flexibility a oddělení konfigurace od samotné logiky alertingu využíváme Cloud Storage bucket, který obsahuje YAML soubor s definicemi alertů. Tento soubor je načítán při každém spuštění alertovací funkce.

Parametr	Hodnota
Název bucketu	Libovolný (např. alerting-bucket)
Region	europe-west3 (Frankfurt)
Storage class	Standard
Access control	Fine-grained (přístup řízený na úrovni objektů)

Data protection	Soft delete (Use default retention duration)
Encryption	Google-managed encryption key (výchozí nastavení)

Dále:

- Pro přístup k objektům v bucketu jsme přiřadili roli `Storage Object Viewer` servisnímu účtu, který byl vytvořen v předchozím kroku. Tím jsme zajistili, že Cloud Run funkce má přístup ke konfiguračnímu souboru
- Byla zapnutá funkce `Object versioning`

Vystavení konfiguračního souboru

Target breadcrumb: `Cloud Storage / Buckets`

V bucketu je uložen soubor `alert_definitions.yaml`, který obsahuje seznam alertovacích pravidel, prahových hodnot a dalších parametrů. Tento soubor je načítán dynamicky při každém běhu služby.

- Název objektu: `alert_definitions.yaml`
- Načítání: pomocí proměnných prostředí `GCS_BUCKET_NAME` a `GCS_BLOB_NAME` v alertovací funkci.

Tento přístup umožňuje měnit alertovací logiku bez nutnosti redeployovat funkci – stačí upravit YAML soubor v bucketu.

Každý blok v YAML souboru definuje jedno alertovací pravidlo a obsahuje následující klíče:

Klíč	Popis
<code>name</code>	Název alertu, používá se např. v logování nebo přehledech.
<code>view</code>	Název BigQuery pohledu (view), který alert sleduje.
<code>message_template</code>	Šablona zprávy, kterou obdrží příjemce. Obsahuje proměnnou <code>{{count}}</code> .
<code>link</code>	URL odkaz na daný Google Sheet přiložený ke zprávě.
<code>recipients</code>	Seznam e-mailových adres, kterým má být alert odeslán.

alert_kind	Typ alertu – "report" nebo "trigger".
enabled	Boolean hodnota určující, zda je alert aktivní (true) nebo ne (false).
threshold	Číselná hodnota – pokud počet záznamů překročí tento práh, alert se spustí.

Kompletní ukázka struktury je na odevzdaném Gitu, soubor se v současném řešení nahrává do bucketu manuálně.

Založení Cloud Run Funkce

Target breadcrumb: Cloud run / Services

Cloud Run služba slouží jako výpočetní vrstva pro alerting nad daty uloženými v BigQuery. Tato služba je navržena tak, aby byla bezpečná, škálovatelná, a efektivní z hlediska nákladů.

Parametry funkce

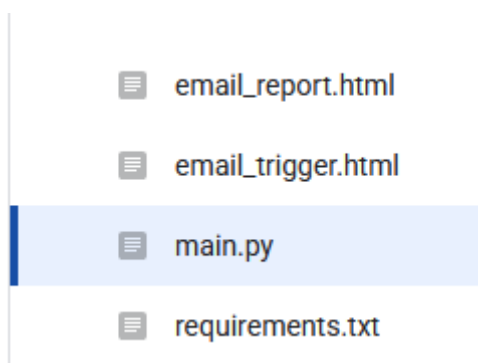
Parametr	Hodnota
Service name	Libovolný (např. alerting)
Region	europe-west3 (Frankfurt)
Runtime	Python 3.12
Authentication	Require authentication (<i>Manage authorized users with IAM</i>)
Billing	Request-based (platba za počet volání)
Service scaling	Automatic scaling
Ingress	Internal only (<i>pouze v rámci GCP sítě</i>)
Service Account	SA založený v předchozích krocích
Environment variables	GCS_BUCKET_NAME, GCS_BLOB_NAME, PROJECT_ID

Google automaticky přiděluje novým funkcím defaultně výchozí cloud run SA. Výměna výchozího service accountu za vlastní servisní účet (vytvořený v předchozích krocích) zajistí, že služba bude mít přesně definovaná oprávnění, což je důležité z hlediska bezpečnosti a auditovatelnosti.

Další parametry služby a provozní detaily

Parametr	Hodnota
Entry point funkce	<code>read_and_alert</code>
Spouštění služby	Pomocí Cloud Scheduleru
Logování	STDOUT výstupy dostupné v Cloud Logging
Timeout / Memory / CPU	Výchozí hodnoty (nezměněno)

Po vytvoření funkce jsou zpřístupněny záložky `source` a `revisions`. Na záložce `source` lze předefinovat obsah funkce²⁴, struktura odpovídá odevzdané verzi na Gitu, funkce vyžaduje následující strukturu:



Cloud Scheduler

Target breadcrumb: `Cloud Scheduler / Jobs`

Alertovací Cloud Run služba je spouštěna automaticky pomocí Google Cloud Scheduleru, který pravidelně odesílá HTTP požadavek na endpoint služby. Díky tomu dochází k pravidelnému zpracování alertů podle definovaných pravidel. Mechanismus určení period je CRON expression. V současné verzi projektu se používají dvě odlišné periody (`report` a `trigger`), které odlišují periodicitu kontroly jednotlivých alert cases, tato perioda defacto definuje typ alertu, a pro každý typ alertu se definuje separátní scheduler úloha.

Každá úloha pak zasílá jiný JSON payload podle typu požadovaného zpracování, v našem případě například:

alerting_report – měsíční reportovací alerty

```
{
  "report_type": "report"
}
```

²⁴ Alternativně lze nastavit CI/CD proces vůči repozitáři.

CRON výraz: `0 8 1 * *` (každého 1. dne v měsíci v 08:00 CEST)

alerting_trigger – týdenní prioritní alerty

```
{
  "report_type": "trigger"
}
```

CRON výraz: `0 9 * * 1` (každé pondělí v 09:00 CEST)

Obě úlohy využívají stejnou službu, ale liší se parametrem `report_type`, který definuje, jaké alerty mají být zpracovány. Různé report types pak používají odlišnou agregační logiku (Trigger se posílá pro každý case zvlášť, report agregovaně).

Konfigurace Scheduler úloh

Parametr	Hodnota
Region	europe-west3
Timezone	Central European Summer Time (CEST)
Target type	HTTP
HTTP method	POST
URL	URL alertovací služby (tj. URL přidělené cloud run funkci v GCP)
HTTP Headers	Content-Type: <code>application/json</code> , User-Agent: <code>Google-Cloud-scheduler</code>
Authentication	Add OIDC token
Service Account	alert-function-sa
Audience	Shodná s URL služby (Cloud Run service URL)
Retry configuration	Výchozí nastavení

Secret Manager

Target breadcrumb: Security / Secret manager

Pro bezpečné uložení API klíčů a dalších citlivých údajů využíváme Google Secret Manager. Pro alertovací Cloud Run funkci byly vytvořeny následující tajemství:

Název tajemství	Účel
-----------------	------

mailgun_api	API klíč pro odesílání e-mailů přes Mailgun
mailgun_domain	Doména Mailgun pro odesílání e-mailů

Obě tajemství jsou uložena ve výchozí konfiguraci (latest version, bez replikace).

Konfigurace mailové proxy

Mailgun by default poskytuje virtuální doménu s limitovanými možnostmi. Pro zprovoznění plné funkcionality je potřeba pracovat na úrovni DNS záznamu. Doména, přes kterou bude přeposlání jednotlivý alert, musí doplnit následující záznamy:

- **CNAME** registrující potřebnou mail proxy doménu (v našem případě: eu.mailgun.org)
- **MX** informace o poštovních serverech (mx1.eu.mailgun.org)
- **TXT** záznam s RSA klíčem
- **TXT** záznam s SPF záznam (Spoofing protection – přihlášení registrovaných/důvěřovaných mailing domény – v našem případě opět mailgun.org)

Konkrétní postupy pro danou doménu lze získat v Mailgun webovém rozhraní. Pro potřeby datového projektu byla použita Google Cloud DNS. V alternativních případech je potřeba, aby záznam přiděl správce domény cílové firmy.

☐ DNS name ↑	Type	TTL (seconds)	Record data
☐ email.mg.martinkadlec.dev.	CNAME	300	eu.mailgun.org.
☐ martinkadlec.dev.	NS	21600	ns-cloud-a1.googledomains.com.
☐ martinkadlec.dev.	SOA	21600	ns-cloud-a1.googledomains.com. cloud-dns-hostmaster.google.com. 1 21600
☐ mg.martinkadlec.dev.	TXT	300	"v=spf1" "include:mailgun.org" "~all"
☐ mg.martinkadlec.dev.	MX	300	10 mx1.eu.mailgun.org.
☐ mta._domainkey.mg.martinkadlec.dev.	TXT	300	"k=rsa,"

Doporučená praxe je pro potřeby mailové komunikace vyhradit (pokud možno) subdoménu (tj. např. alerting.behaviolabs.com).

4.2.2 NLQ

Příprava projektu

Oproti výše popsanému postupu jsou potřeba dodatečné celo-projektové aktivace:

Povolení potřebných API:

- Vertex AI API

Dále implementační osoba musí mít přidělené role:

- Artifact Registry Administrator
- BigQuery Admin
- Cloud Build Editor
- Cloud Run Admin
- OAuth Config Editor
- Private Logs Viewer
- Project IAM Admin
- Service Account User
- Service Usage Admin
- Storage Admin

Dále je potřeba analogicky podle postupu výše (a z totožných důvodů) připravit dedikovaný servisní účet.

Potřebné role pro SA:

Role	Účel
<code>roles/bigquery.dataViewer</code>	Čtení dat z BigQuery
<code>roles/vertexAI.user</code>	Přístup k VertexAI compliant modelům
<code>roles/bigquery.jobUser</code>	Umožňuje spouštět joby (typicky queries) nad BQ.
<code>roles/bigquery.user</code>	Poskytuje přístup k BQ tabulkám a metadatům.
<code>roles/bigquery.readSessionUser</code>	Umožňuje vytvářet a číst BQ sessions.

Příprava dat

Target breadcrumb: `BigQuery`

Za předpokladu, že jsou data připravena v očekávaném formátu (tj. prošla pre-processingem), nejsou potřeba další úpravy. Chatbot v rámci své konfigurace (`tables.yaml`) dostává předpis struktury dat a tabulek, na které je možné se dotazovat.

V opačném případě je potřeba připravit BQ tables ve vhodné datové kvalitě – pro správný chod bota jsou vhodné minimálně správně natypované sloupce a reprezentativní názvy sloupců.

Příprava Endpointu

Target breadcrumb: `Cloud run / Services`

Kontejnerový deployment umožňuje provést initial deploy s dummy aplikací, toto je důležité, protože je v tomto kroku vystaven Endpoint URL, která se používá pro konfiguraci OAuth zabezpečení.

Během procesu založení service je potřeba specifikovat očekávané jméno služby, a poté použít možnost test (případně lze projít deploy procesem dvakrát) – tímto se služba instancuje s ukázkovým kontejnerem webové služby, ale vystaví se potřebné endpointy pro další kroky.

Nastavení OAuth zabezpečení

V aplikaci je přibalena OAuth zabezpečovací vrstva, kterou je potřeba konfigurovat spolu se servisem běžícím v GCP. Tato konfigurace má dva kroky.

Prvním z nich je nastavení OAuth Screen.

Target breadcrumb: Google Auth Platform / Branding

V rámci OAuth screenu je potřeba vyplnit pole AppName a User Support Email, AppName je jméno, které se zobrazuje uživatelům při přihlášení, a daný email je pak v daném loginu uveden jako kontaktní osoba. Na konci stránky je pak jako Authorized domains potřeba uvést EndpointURL (či chcete-li service URL) service připravené v předchozím kroku.

Authorized domains

When a domain is used on the consent screen or in an OAuth client's configuration, it must be pre-registered here. If your app needs to go through verification, please go to the [Google Search Console](#) to check if your domains are authorized. [Learn more](#)  about the authorized domain limit.

Authorized domain 1 *
behavio-bot-749895389873.europe-west3.run.app

Authorized domain 2 *
behavio-bot-749895389873.europe-west4.run.app

[+ Add domain](#)

Developer contact information

Email addresses *

[Redacted email address]

Jako poslední povinné pole je pak potřeba vyplnit kontaktní mail správce aplikace (doporučujeme uvést správce instance), na tento mail pak chodí notifikace spojené s případným provozem OAuth Clienta.

Target breadcrumb: Google Auth Platform / Clients

Druhým krokem je pak založení tzv. OAuth Klienta. Na záložce clients se založí nový klient s typem webové aplikace. Na názvu klienta nezáleží, slouží pro identifikaci klienta v rámci GCP.

Klíčové je, aby používala URL service jako trusted JavaScript Origin (jinak se vrátí http 403) a aby Authorized redirect URI ukazovala zpět na tento origin (jinak po doběhnutí autentizace nedojde k přesměrování zpět do aplikace). Po založení klienta je potřeba ještě vytvořit secret, který se používá v dalším kroku jako konfigurační parametr.

Authorized JavaScript origins ⓘ
For use with requests from a browser

URIs 1 *
<https://behavio-bot-749895389873.europe-west4.run.app>

+ Add URI

Authorized redirect URIs ⓘ
For use with requests from a web server

URIs 1 *
<https://behavio-bot-749895389873.europe-west4.run.app>

+ Add URI

Client secrets
Inactive OAuth clients are subject to deletion if they are not used for 6 months. You will be notified of deletion due to inactivity, and can restore clients up to 30 days after deletion. [Learn more](#)

If you are in the process of changing client secrets, you can manually rotate them without downtime. [Learn more](#)

Client secret	[REDACTED]
Creation date	May 17, 2025 at 7:33:40 PM GMT+2
Status	Enabled

+ Add secret

Příprava repozitáře

Současný deployment režim je integrován skrze lokální stanici implementátora²⁵.

Je potřeba připravit `.env` file v následující struktuře:

```
GOOGLE_OAUTH_CLIENT_ID=xxxxxxxxx.apps.googleusercontent.com
GOOGLE_OAUTH_CLIENT_SECRET=xxxxxxxxx
GOOGLE_OAUTH_REDIRECT_URI=https://behavio-chatbot-xxxx.a.run.app
GCP_PROJECT_ID=your-gcp-project
GCP_VERTEX_LOCATION=europe-west3
GOOGLE_APPLICATION_CREDENTIALS=/secrets/gcp-credentials.json
WHITELISTED_EMAILS=uzivatel1@firma.cz,uzivatel2@firma.cz
WHITELISTED_DOMAINS=firma.cz
```

Je vhodné držet na paměti, že obsah `.env` file je bezpečnostně citlivý a neměl by být propagován veřejně (např. na veřejný Git).

OAuth parametry odpovídají své protistraně z předchozího kroku.

GCP project ID je unikátní identifikátor instance (součástí URL).

²⁵ Viz. Assessment kompromisů v tech stacku a Doporučení pro další rozvoj. Realisticky v dokumentaci momentálně mimo automatizované CI/CD neexistuje vhodnější režim nasazení.

Vertex Location je požadovaný zdroj modelů a google application credentials jsou private key pro daný servisní účet.

Whitelisted Emails a Domains pak umožní těmto účtům přistupovat k aplikaci.

Json klíč servisního účtu v rámci GCP se vkládá do samostatného souboru projektu `credentials/credentials.json`. Tento klíč zajišťuje autentizaci a autorizaci chodu SA.

Dalším velmi důležitým krokem je příprava `tables.yaml` souboru. Tento soubor předepisuje kontext pro chatbota – na jaké tabulky se má dotazovat a co je jejich obsahem. Námi testovaná struktura může vypadat např. následovně:

```
tables:
- name: dwh_develop.synth_invoices
  description: Tabulka obsahuje všechny vystavené faktury, jejich částky, splatnosti a
stav zaplacení.
  schema: |
    | invoice_id | STRING | Unikátní identifikátor faktury (např. "INV-0001") |
    | customer_id | STRING | Unikátní identifikátor zákazníka |
    | customer_name | STRING | Jméno zákazníka |
    | issue_date | DATE | Datum vystavení |
    | due_date | DATE | Datum splatnosti |
    | amount | FLOAT | Částka faktury |
    | currency | STRING | Měna (např. "CZK", "USD") |
    | status | STRING | "Paid", "Open", "Upcoming Due", nebo "Overdue" |
    | paid_date | DATE | Datum zaplacení (NULL pokud nezaplaceno) |
    | created_at | TIMESTAMP | Čas vytvoření záznamu |
    | updated_at | TIMESTAMP | Čas poslední aktualizace |
```

Realisticky je celý problém o poskytnutí sémantické reprezentace dat, není striktně nutné dodržovat tento formát, měl by posloužit libovolný data profile výstup převedený do .yaml formátu.²⁶ V námi zvolené struktuře je jedna tabulka podchycena jako jeden objekt, tento objekt lze libovolně rozšířit dle potřeby a požadavků na další rozvoj.

Deploy

Lze testovat lokálně pomocí²⁷:

```
docker build -t behavio-bot .
docker run -p 8080:8080 --env-file .env behavio-bot
```

²⁶ Automatizace tohoto kroku je součástí doporučení pro další rozvoj.

²⁷ Vyžaduje Docker Engine na lokální stanici. Pro lokální testování je potřeba přepnout OAuth autorizace na localhost:8080, jinak bude mít služba tendence redirektovat na aktivní instanci běžící v GCP.

Build se však dá nechat provést přímo v GCP²⁸:

```
gcloud builds submit --tag gcr.io/TVUJ_PROJECT_ID/behavio-bot

gcloud run deploy behavio-bot \
--image gcr.io/TVUJ_PROJECT_ID/behavio-bot \
--service-account=TVUJ_SERVICE_ACCOUNT@TVUJ_PROJECT_ID.iam.gserviceaccount.com \
--platform managed \
--region europe-west3 \
--allow-unauthenticated
```

V době provedení buildu dojde k „zapouzdření“ obsahu `.env` file a `tables.yaml` do obsahu aplikační vrstvy.

Post deploy checks

- Otevřete nasazenou adresu služby (např. <https://behavio-chatbot-xxxx.a.run.app>)
- Přihlaste se Google účtem z whitelistu/domény.
- Vyzkoušejte zadání dotazu a ověřte komunikaci s BigQuery i Vertex AI.
- Prohlédněte logy služby – buď v console – nebo pomocí:

```
gcloud run services logs read behavio-bot
```

4.3 Doporučení pro další rozvoj

K datu sepsání této kapitoly (15.06.2025) je projekt ve stavu kdy:

- Bylo provedeno on-site UAT za přítomnosti řešitelského týmu, Lukáše Tótha, Ho Phuong Anh a Miroslava Umlaufa. (viz. Cílové skupiny a účastníci)
- Na základě výstupů z UAT byly provedeny úpravy v aplikační vrstvě a předány k re-testu
- Lukáš Tóth potvrdil očekávané zlepšení funkcionality
- Lukáš Tóth potvrdil zájem používat obě dodané komponenty v chodu firmy

²⁸ Využívá gcloud CLI lokální knihovnu. Je potřeba provést login účtem, který má požadované IAM role. Viz. <https://cloud.google.com/sdk/gcloud>. Provedení deploye tímto způsobem automaticky provede předepsané změny dle obsahu příkazu. Je vhodné držet na paměti že obsah ENV file je destruktivní vůči předchozím konfiguracím, je tedy vhodné důsledně kontrolovat, že především OAuth proměnné jsou nastaveny správně. Alternativně lze vynechat run deploy krok, a provést revizi v online konzoli s odkazem na latest tag.

Prvním krokem pro PROD běh komponent by měl být replikace výstupů na PROD instanci. Za tímto účelem lze sledovat Implementační postup. Jediná aktivní proměnná je volba mailing proxy. Lze buď otevřít novou Mailgun instanci a registrovat ji pod DNS domény Behavio, nebo projít konfiguračním procesem pro GmailAPI v rámci Google Workspace.

Doporučením řešitelského týmu je v průběhu 1-2 měsíců sledovat objem odesílaných mail zpráv a vytížení Cloud Run billing costs. V případě, že všechny metriky jsou v normě, je řešení ve své podstatě implementováno. Dokumentace založení postupu nového alertu je součástí projektové Wiki. Kodifikace rozvoje datového setu pro chatbota se hůře standardizuje a je zde přípustná navazující diskuse. V současném řešení jsou metadata připravena ručně. Toto pochopitelně není ideální řešení z hlediska dlouhodobé škálovatelnosti. Možné řešení bez integrace současných data management řešení v GCP je připravit skript, který by prováděl kontrolu a export metadat pomocí vlastního analytického řešení na úrovni Python Cloud Run funkce. Správné řešení problému zde musí být podřízeno očekávanému objemu a rychlosti přírůstku nových tabulek do DWH.

U NLQ řešení řešitelský tým doporučuje optimistickou skepsi a sledovat vývoj na poli GCP novinek. Google v rozmezí posledních týdnů ohlásil, že s vybranými cloud partnery testují rozšíření dosavadně dostupných analytických možností v podstatě pro všechna dostupná databázová řešení. Bohužel dnešní optikou nelze určit, k jakému datu dojde k nějakému významnému ohlášení, ale lze očekávat, že ještě letos k nějakému vývoji dojde. Z tohoto důvodu bychom jako primární účel NLQ řešení v současném stavu viděli testování možností a přínosů tohoto přístupu²⁹. Toto ohlášení nemusí být nutně vnímáno jako kompletní náhrada dodané komponenty, je možné, a dokonce poměrně pravděpodobné, že v řešeních budou jisté odchylky.

4.4 Technická rozvaha pricing jednotek

Jedním z nejvíce diskutovaných a *diskutabilně* proměnlivých bodů projektu je konkrétní vyjádření nákladové funkce, která je vyvolána implementací nových komponent do prostředí GCP.

V podstatě každá komponenta používá svůj vlastní výpočet nákladů, který je vhodné zvážit.

Cloud Scheduler

Bez větších komplikací, pro každý CRON job nad 3 joby se platí 10 centů měsíčně. Projekt v současném provedení používá dva joby.

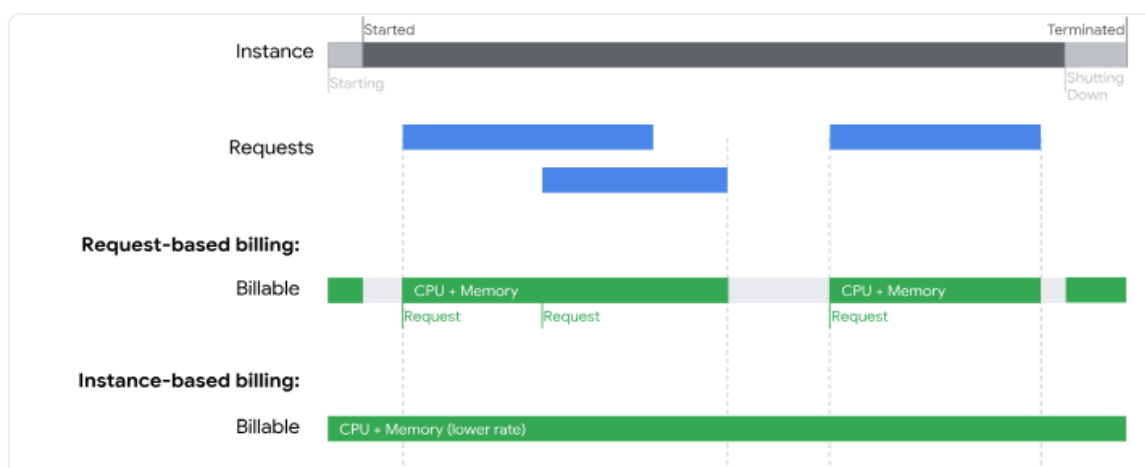
²⁹ Realisticky lze očekávat, že under-the-hood mechanismus bude nějaká forma Gemini či Gemma modelu, hlavní rozdíly lze čekat právě v obsluze kontextu a orchestraci chodu LLM agentů. Pokud uvažujeme v rovině, že lze vycházet z představení z letošního NEXT a GC Summitu, ambice Google je s největší pravděpodobností nějaká forma multi-agentního systému.

Pricing table

Job cost	Free tier
\$0.10 per job per month	3 free jobs per month, per billing account

Cloud Run

GCP umožňuje dva billing režimy fakturace, tzv. Request based billing vs. Instance based billing.



Zjednodušeně řečeno, v instance-based systému je uživateli fakturován konstantní (ale snížený) paušální poplatek daný výpočetním vybavením přiřazených kontejnerů. Oproti tomu v request-based systému je uživateli fakturován pouze čas stráven obsluhou samotných requestů a čas který je stráven během startování a uspávání kontejnerů.

Request-based billing je tedy zvolenou variantou pro potřeby tohoto projektu pro obě dodané komponenty. Očekávaný workload je ve velmi nízkých řádech³⁰ a v žádném případě nekvalifikuje instance-based přístup jako vhodnou volbu³¹.

Google dále rozlišuje tzv. Tier 1 a Tier 2 pricing regiony. Tier 1 regiony mají levnější SKUs a mají větší free plán. V tieringu není geografická logika – jedná se o reflexi provozních nákladů datacentra (např. Belgie je Tier 1, Frankfurtské datacentrum je Tier 2) – celá problematika je dále zkomplikována tím, že Google jako fakturační jednotku používá kromě počtu requestů také jejich jednotlivý exekuční čas. Volba datacentra dále ovlivňuje dostupné komponenty (typicky VertexAI modely). Doporučením řešitelského týmu – pokud není

³⁰ Request based billing efektivně fakturuje 0 USD v časech, kdy instance nejsou využívány. Toto je použitá fakturační jednotka v předaném prostředí.

³¹ Zajímavostí může být, že Google momentálně nepodporuje request billing pro instance opatřené GPU. Důvodem jsou architekturní rozdíly mezi CPUs a GPUs, které nedovolují „uspávat“ GPU stejným způsobem, jak general purpose výpočetní jednotky.

potřeba brát v potaz nějaký konkrétní důvod – je volit nejbližší dostupné datacentrum, tím je zajištěna minimalizace latence, a tedy i očekávaných exekučních časů.

Location	Region	Median Latency ↓
Frankfurt	europa-west3	43 ms
Global External HTTPS Load Balancer	→europa-west3	44 ms
Warsaw	europa-central2	47 ms
Zurich	europa-west6	50 ms
Belgium	europa-west1	51 ms
Netherlands	europa-west4	51 ms
Milan	europa-west8	55 ms
Turin	europa-west12	56 ms
London	europa-west2	57 ms
Berlin	europa-west10	58 ms
Paris	europa-west9	59 ms
Stockholm	europa-north2	69 ms

Současný tier 2 CPU pricing k datu vypracování této kapitoly (23.06.2025) je po přepočtu 35,714 hodiny čistého výpočetního času. Tier 1 Pricing poskytuje 50 hodin čistého výpočetního času. Nejbližší Tier 1 Datacentra jsou buď v Belgii anebo ve Stockholmu. Nejbližší Tier 2 centra jsou ve Frankfurtu, Zurichu a Varšavě.

Během chodu komponent dochází ke generaci přidruženého **Artifact Registry** billingu. Jedná se o poplatky spojené s obsluhou docker images a buildy aplikací. Řádově se bavíme však o jednotkách korun měsíčně. Aplikace mohou dále generovat tzv. **Networking** a **Storage** costs. Jedná se o poplatky pramenící z obsluhy komunikace přes HTTPS/IP a provoz bucketů držící konfigurační soubory – zde se bavíme o halířích měsíčně.³²

VertexAI

Projekt používá možnosti Gemini 2.0 Flash. Současná fakturační politika předepisuje rozdílné ceny pro Input / Output tokeny.

³² V případech, kdy je doména obsluhující mailovou komunikací registrována pod GCP, vyskytnou se také DNS costs. Opět v řádech halířů až korun.

Model	Type	Price	Price with Batch API
Gemini 2.0 Flash	1M Input tokens	\$0.15	\$0.075
	1M Input audio tokens	\$1.00	\$0.50
	1M Output text tokens	\$0.60	\$0.30
	Tuning for 1M training tokens	\$3.00	

Google ve své vývojářské dokumentaci neposkytuje přesnou rate pro kalkulaci textových tokenů, pravděpodobně kvůli tomu, že různé jazyky používají různé tokenizace, a protože algoritmus pro jejich určení není v čase stabilní. Předpokládejme, že průměrně platí konverze 0.5 slova / token, tímto odhadem dostáváme, že jedna fakturační jednotka, tj. milion tokenů, odpovídá 1 500 000 slovům³³.